Technical Report

War Games: Monte Carlo Simulations and the Likelihood of Victory

Senior Capstone Project

Subject: Monte Carlo battle simulation and victory likelihood estimation

Application: Capstone Project – Clonlara School

Prepared by: Thomas X. Valenzuela

Advisor: Dr. John R. Valenzuela

Date: September 7, 2026

Version: Draft v0.1

Main idea. This project estimates the probability of victory in a rule-based war game by treating uncertain battle events as random variables and evaluating many simulated battles. The purpose of the report is explain how Monte Carlo simulations can be used to inform strategy in the board game *1754: Conquest*

Abstract

[TODO: Write a concise 150–300 word abstract after the results are complete. Include the problem, simulation method, number of trials, most important numerical result, and final conclusion.]

Example structure: This report develops a Monte Carlo simulation for estimating the likelihood of victory in a turn-based war game. Each battle is represented as a probabilistic experiment whose outcome depends on unit strengths, terrain, rule modifiers, and random combat resolution. The simulation is implemented in MATLAB and repeated over many trials to estimate victory probability, expected losses, and sensitivity to strategic choices. Results are summarized with confidence intervals and used to identify robust strategies.

Contents

Abstract 1

1 Executive Summary 4

2 Purpose and Scope 5

2.1 Research Questions 5

2.2 Deliverables 5

3 Background and Theory 5

3.1 Why Monte Carlo Simulation? 5

3.2 Game Model Overview 6
3.3 Outcome Metrics 6

4 Assumptions, Definitions, and Notation 7
4.1 Assumptions 7
4.2 Notation 7
4.3 Scenario Parameter Table 8

5 Simulation Methodology 8
5.1 High-Level Execution Flow 8
5.2 One-Trial Battle Logic 8
5.3 Monte Carlo Estimation 8

6 MATLAB Implementation 9
6.1 Code Organization 9
6.2 Insert MATLAB Code Here 9
6.3 Implementation Notes 10

7 Verification and Validation 10
7.1 Verification Checks 11
7.2 Convergence Check 11

8 Experimental Design 11
8.1 Baseline Scenario 11
8.2 Alternative Strategies 11

9 Results 12
9.1 Victory Probability 12
9.2 Outcome Distributions 12

10 Sensitivity and Uncertainty Analysis 12

11 Discussion 13

12 Limitations and Threats to Validity 13

13 Reproducibility Checklist 13

14 Conclusions and Future Work **14**

References **14**

A MATLAB Source Code Appendix **15**

B Detailed Parameter Tables **15**

C Additional Figures **16**

D Derivation of the Confidence Interval **16**

List of Figures

1 this is my Caption text. 4
2 Recommended high-level workflow for the Monte Carlo battle simulation. 8
3 Placeholder for Monte Carlo convergence plot. 11
4 Placeholder for battle outcome distribution. 12

List of Tables

1 Primary simulation metrics. 7
2 Notation used in the report. 7
3 Example scenario parameter table. Replace the entries with the final game parameters. 8
4 Recommended MATLAB source-file organization. 9
5 Recommended verification checklist. 11
6 Example experimental design matrix. 12
7 Template for final probability-of-victory results. 12
8 Template for sensitivity analysis results. 13
9 Reproducibility checklist. 14
10 Full parameter table template. 16

1 Executive Summary

This project was developed after my father and I lost a battle in the board game *1754: Conquest* to my younger sister and younger brother. The battle seemed like it was nearly impossible to lose, the imbalance of the size of forces should have ensured our victory, but it didn't. After the game was over, my mentor (which is my father) suggested that I create a simulation in MATLAB to recreate the battle and compute what the likelihood of victory actually was. After learning that rolling dice can be accurately simulated by random number generators, I set out to write code in MATLAB that could simulate an arbitrary battle in the game.

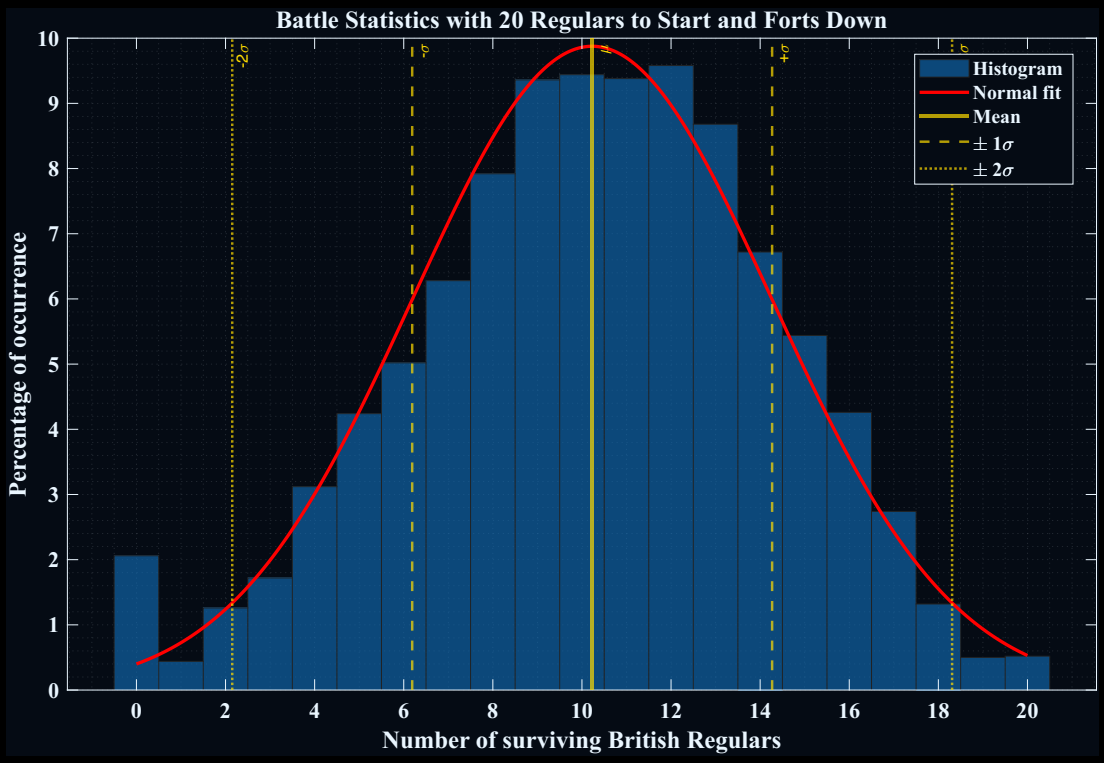

Figure 1: this is my Caption text.

The code accommodates all possible input parameters for a battle, all associated rules for a specific battle, and is flexible enough to perform trade studies over input parameters. In addition to the core simulator, I wrote analysis code that performs trade studies and analyzes the statistical distributions of the results as well as rates of change of output parameters as a function of input parameters.

Example outputs of the code shown in Figure 1.

NOW DESCRIBE THE BATTLE THAT YOUR FIGURE IS FOR

- **Problem.** Estimate battle-level victory probabilities for a rule-based war game.
- **Method.** Build a MATLAB Monte Carlo simulation that repeats the same scenario many times while sampling the uncertain parts of the game.
- **Primary outputs.** Victory probability, expected surviving forces, loss distributions, and sensitivity to assumptions.
- **Key result.** **[TODO: Insert final numerical result, for example: Strategy A wins $\widehat{p} = 0.67$ of trials with a 95% confidence interval of $[0.65, 0.69]$.]**
- **Recommendation.** **[TODO: State the preferred strategy and the evidence supporting it.]**

2 Purpose and Scope

This report documents the mathematical model, MATLAB implementation, validation tests, and results for a Monte Carlo war-game simulation. The scope is limited to the battle-resolution model described in this report. Political, economic, and psychological factors are outside scope unless they are explicitly represented by game rules or input parameters.

2.1 Research Questions

1. What is the estimated probability that a specified force wins a specified battle?
2. How many surviving units should each side expect after the battle?
3. Which assumptions or game parameters most strongly change the victory probability?
4. Are any strategies consistently better across many plausible assumptions?

2.2 Deliverables

- A documented MATLAB simulation.
- A reproducible set of experiments and random seeds.
- Tables and figures summarizing probability of victory, expected losses, and uncertainty.
- A technical report explaining the modeling assumptions, validation checks, and conclusions.

3 Background and Theory

3.1 Why Monte Carlo Simulation?

A Monte Carlo simulation estimates a quantity by repeatedly sampling random outcomes from a model. In this project, one simulated battle is one random trial. After N_{trials} independent trials, the estimated

probability of victory is

$$\widehat{p} = \frac{1}{N_{\text{trials}}} \sum_{i=1}^{N_{\text{trials}}} I_i, \tag{1}$$

where $I_i = 1$ if the selected side wins trial i and $I_i = 0$ otherwise.

For a binary win/loss outcome, an approximate standard error is

$$\text{SE}(\widehat{p}) = \sqrt{\frac{\widehat{p}(1-\widehat{p})}{N_{\text{trials}}}}. \tag{2}$$

A rough 95% confidence interval is therefore

$$\widehat{p} \pm 1.96\,\text{SE}(\widehat{p}). \tag{3}$$

This interval measures Monte Carlo sampling uncertainty, not uncertainty caused by incorrect assumptions in the model.

3.2 Game Model Overview

The battle model should be described in enough detail that another person could implement it independently. Important ingredients usually include:

- unit counts and unit types,
- attack and defense values,
- terrain, fortification, or morale modifiers,
- random dice rolls or probability tables,
- turn order and stopping conditions,
- victory conditions.

3.3 Outcome Metrics

The most important scalar output is the probability of victory. Additional metrics are useful because two strategies may have similar win probability but very different cost.

Table 1: Primary simulation metrics.

Metric	Meaning
Victory probability	Fraction of trials won by the selected force.
Expected survivors	Mean number of units remaining at the end of the battle.
Expected losses	Mean number of units lost during the battle.
Median outcome	Typical outcome, less sensitive to rare extreme trials.
Risk of severe loss	Probability that losses exceed a specified threshold.
Simulation runtime	Time required to run the selected number of trials.

4 Assumptions, Definitions, and Notation

4.1 Assumptions

Assumption log. Keep this section explicit. A technical report is stronger when the reader can separate results caused by the data from results caused by modeling choices.

1. Each trial is independent of the others once the scenario parameters are fixed.
2. The random number generator is initialized with a documented seed for reproducibility.
3. The rules implemented in MATLAB match the written game rules.
4. Player decisions within a strategy are represented by deterministic rules or documented random policies.
5. Any omitted game effects are either negligible or outside the stated scope.

4.2 Notation

Table 2: Notation used in the report.

Symbol	Meaning
N_{trials}	Number of Monte Carlo trials.
I_i	Win indicator for trial i.
$\widehat{p}$	Estimated probability of victory.
S_A, S_B	Surviving units for forces A and B.
L_A, L_B	Losses for forces A and B.
θ	Vector of scenario parameters.
$\mathrm{SE}(\widehat{p})$	Standard error of estimated win probability.

4.3 Scenario Parameter Table

Table 3: Example scenario parameter table. Replace the entries with the final game parameters.

Parameter	Symbol	Baseline value	Notes
Attacking infantry	$N_{A,\text{inf}}$	[TODO: value]	Number of attacking infantry units.
Defending infantry	$N_{B,\text{inf}}$	[TODO: value]	Number of defending infantry units.
Fortification modifier	m_{fort}	[TODO: value]	Defensive multiplier or rule modifier.
Terrain modifier	m_{terrain}	[TODO: value]	Terrain effect applied during combat.
Number of trials	N_{trials}	[TODO: value]	Use enough trials for stable estimates.
Random seed	–	[TODO: value]	Required for reproducibility.

5 Simulation Methodology

5.1 High-Level Execution Flow

Figure 2: Recommended high-level workflow for the Monte Carlo battle simulation.

5.2 One-Trial Battle Logic

Describe the rule sequence for a single trial. A good format is:

1. Initialize unit counts and modifiers.
2. Draw random rolls needed for the current battle step.
3. Apply attack, defense, terrain, and special-rule effects.
4. Update casualties and survivors.
5. Check victory or stopping conditions.
6. Return trial outputs: winner, survivors, losses, number of turns, and any diagnostic values.

5.3 Monte Carlo Estimation

After one-trial logic is correct, the full simulation is a loop over trials. Store raw outcomes whenever possible. Raw output enables later plotting, debugging, and sensitivity analysis without rerunning every trial.

$$\text{trial output}_i = f(\theta, U_i), \tag{4}$$

where θ is the fixed scenario parameter vector and U_i represents the random numbers sampled during trial i.

6 MATLAB Implementation

This section is reserved for MATLAB code, implementation details, and reproducibility notes. The code block below can be replaced with the final simulation driver.

6.1 Code Organization

Recommended file organization:

Table 4: Recommended MATLAB source-file organization.

File	Role
`runCapstoneSimulation.m`	Main script that defines scenarios, calls the simulator, and generates plots.
`simulateBattleMonteCarlo.m`	Runs N_{trials} independent trials and returns summary statistics.
`simulateOneBattle.m`	Implements one battle from initialization to victory condition.
`summarizeBattleResults.m`	Converts raw trial outputs into probabilities, confidence intervals, and tables.
`plotBattleResults.m`	Generates report-quality figures.

6.2 Insert MATLAB Code Here

```
function results = simulateBattleMonteCarlo(params)
%SIMULATEBATTLEMONTECARLO Estimate victory probability by Monte Carlo.
%
% INPUT
%   params.nTrials     - number of Monte Carlo trials
%   params.randomSeed  - random seed for reproducibility
%   params.scenario    - struct containing units, modifiers, and rules
%
% OUTPUT
%   results            - struct containing raw outcomes and summary metrics

rng(params.randomSeed,'twister');

nTrials = params.nTrials;
winner  = strings(nTrials,1);
survA   = zeros(nTrials,1);
survB   = zeros(nTrials,1);

for t = 1:nTrials
    trial = simulateOneBattle(params.scenario);

```

```
    winner(k) = trial.winner;
    survA(k)  = trial.survivorsA;
    survB(k)  = trial.survivorsB;
end

winA = winner == "A";
pHat = mean(winA);
se   = sqrt(pHat*(1 - pHat)/nTrials);
ci95 = pHat + 1.96*se*[-1 1];

results = struct();
results.winner = winner;
results.survivorsA = survA;
results.survivorsB = survB;
results.pVictoryA = pHat;
results.pVictoryA_CI95 = ci95;
results.expectedSurvivorsA = mean(survA);
results.expectedSurvivorsB = mean(survB);
end
```

Listing 1: Template MATLAB driver for Monte Carlo battle simulation.

To include a full external MATLAB file instead of pasting code directly, use:

```
% Put this command in the LaTeX report, not inside MATLAB:
% \lstinputlisting[style=matlabstyle,caption={Main simulation driver.}]{runCapstoneSimulation.m}
```

Listing 2: Example of including an external MATLAB source file.

6.3 Implementation Notes

Document important implementation choices here:

- random seed and random-number stream,
- vectorized versus loop-based implementation,
- how game rules are represented in code,
- how ties or edge cases are handled,
- MATLAB version and toolbox dependencies,
- any functions that were tested separately.

7 Verification and Validation

Verification asks whether the code was implemented correctly. Validation asks whether the implemented model is appropriate for the real game question.

7.1 Verification Checks

Table 5: Recommended verification checklist.

Check	Description	Status
Deterministic seed	Same seed gives identical results.	[TODO: Pass/Fail]
Known trivial case	Overwhelming force wins nearly all trials.	[TODO: Pass/Fail]
Symmetry case	Identical armies have approximately equal win probability.	[TODO: Pass/Fail]
Conservation check	Survivors plus losses equal starting units.	[TODO: Pass/Fail]
Rule audit	MATLAB logic matches written rules.	[TODO: Pass/Fail]

7.2 Convergence Check

Run the simulation with increasing N_{trials} and verify that $\widehat{p}$ stabilizes. A convergence plot should show estimated win probability versus number of trials.

Insert convergence plot here.
Example: $\widehat{p}$ versus number of trials with a 95% confidence band.

Figure 3: Placeholder for Monte Carlo convergence plot.

8 Experimental Design

The experiments should be defined before looking at final outcomes. This avoids choosing only the experiments that support a preferred conclusion.

8.1 Baseline Scenario

State the baseline battle configuration and why it was selected.

8.2 Alternative Strategies

List strategies or rule settings to compare. Each strategy should differ by one clearly defined decision or parameter group.

Table 6: Example experimental design matrix.

Experiment	Strategy / condition	Changed parameter	Purpose
Baseline	[TODO: strategy]	None	Reference case.
Experiment 1	[TODO: strategy]	[TODO: parameter]	Test tactical change.
Experiment 2	[TODO: strategy]	[TODO: parameter]	Test fortification effect.
Experiment 3	[TODO: strategy]	[TODO: parameter]	Test troop composition.

9 Results

This section should contain the most important tables and figures. Put detailed or secondary plots in the appendix.

9.1 Victory Probability

Table 7: Template for final probability-of-victory results.

Scenario	Trials	$\widehat{p}$	95% CI	Expected survivors
Baseline	[TODO: N]	[TODO: value]	[TODO: interval]	[TODO: value]
Strategy 1	[TODO: N]	[TODO: value]	[TODO: interval]	[TODO: value]
Strategy 2	[TODO: N]	[TODO: value]	[TODO: interval]	[TODO: value]

9.2 Outcome Distributions

Insert histogram or distribution plot here.
Example: surviving units, losses, or number of turns.

Figure 4: Placeholder for battle outcome distribution.

10 Sensitivity and Uncertainty Analysis

Sensitivity analysis asks how much the conclusion changes when assumptions change. This is especially important for a simulation because a precise answer from a poor assumption can be misleading.

Table 8: Template for sensitivity analysis results.

Parameter varied	Low value	Baseline	High value	Effect on conclusion
Fortification modifier	[TODO:]	[TODO:]	[TODO:]	[TODO:]
Attacker strength	[TODO:]	[TODO:]	[TODO:]	[TODO:]
Defender strength	[TODO:]	[TODO:]	[TODO:]	[TODO:]
Random event probability	[TODO:]	[TODO:]	[TODO:]	[TODO:]

11 Discussion

Interpret the results rather than repeating them. Useful questions:

- Which result was most expected?
- Which result was surprising?
- Did higher win probability come with higher expected losses?
- Are the differences between strategies practically meaningful or just statistically detectable?
- What does the simulation suggest about good play?

12 Limitations and Threats to Validity

1. **Rule simplification.** Some real or game-specific effects may be simplified or omitted.
2. **Input uncertainty.** Results depend on selected unit strengths, modifiers, and assumptions.
3. **Randomness model.** The random draws must match the actual game mechanics.
4. **Strategy representation.** A strategy implemented as code may not capture all human decision-making.
5. **Monte Carlo error.** Finite trial counts introduce sampling uncertainty, even if the model is correct.

13 Reproducibility Checklist

A reader should be able to reproduce the main numerical results from the information in this section.

Table 9: Reproducibility checklist.

Item	Required information	Included?
MATLAB version	Version and installed toolboxes.	[TODO: Yes/No]
Random seed	Seed used for each reported experiment.	[TODO: Yes/No]
Scenario parameters	Unit counts, modifiers, and rule options.	[TODO: Yes/No]
Source code	Main scripts and helper functions.	[TODO: Yes/No]
Generated data	Saved raw trial outputs or summary tables.	[TODO: Yes/No]
Figure scripts	Code that regenerates each report figure.	[TODO: Yes/No]

14 Conclusions and Future Work

Summarize the project in one or two paragraphs. Include the most important numerical result, the best-supported interpretation, and the next most useful improvement.

Potential future work:

- add more unit types or special rules,
- compare more strategies,
- optimize strategies automatically,
- build an interactive MATLAB app,
- validate the simulation against hand-calculated examples or historical game outcomes.

References

Use this section for books, articles, game manuals, MATLAB documentation, or other sources. Replace the placeholder references below with final sources.

References

[1] N. Metropolis and S. Ulam, "The Monte Carlo Method," *Journal of the American Statistical Association*, vol. 44, no. 247, pp. 335–341, 1949.

[2] MathWorks, *MATLAB Documentation*. Natick, MA: The MathWorks, Inc.

[3] [TODO: Add the war-game rulebook or source document used for the simulation.]

A MATLAB Source Code Appendix

Place final source files here or include them with `\lstinputlisting`. For long projects, include only the most important functions in the report and provide the full code repository separately.

```
function trial = simulateOneBattle(scenario)
%SIMULATEONEBATTLE Resolve one randomized battle.
%
% Replace this skeleton with the final game logic.

unitsA = scenario.unitsA;
unitsB = scenario.unitsB;

while unitsA > 0 && unitsB > 0
    rollA = randi(6);
    rollB = randi(6);

    attackScore  = unitsA + rollA + scenario.attackModifier;
    defenseScore = unitsB + rollB + scenario.defenseModifier;

    if attackScore > defenseScore
        unitsB = unitsB - 1;
    elseif defenseScore > attackScore
        unitsA = unitsA - 1;
    else
        % Tie rule: no losses in this placeholder model.
    end
end

if unitsA > unitsB
    trial.winner = "A";
elseif unitsB > unitsA
    trial.winner = "B";
else
    trial.winner = "Tie";
end

trial.survivorsA = unitsA;
trial.survivorsB = unitsB;
end
```

Listing 3: Skeleton one-battle simulator.

B Detailed Parameter Tables

Use this appendix for complete parameter listings that are too detailed for the main text.

Table 10: Full parameter table template.

Parameter	Value	Source / note
[TODO: parameter]	[TODO: value]	[TODO: source]
[TODO: parameter]	[TODO: value]	[TODO: source]
[TODO: parameter]	[TODO: value]	[TODO: source]

C Additional Figures

Place extra plots here: histograms, convergence diagnostics, sensitivity curves, and scenario diagrams.

D Derivation of the Confidence Interval

For a fixed scenario, each trial win indicator I_i is modeled as a Bernoulli random variable with success probability p. The sample mean $\widehat{p}$ estimates p. The variance of a Bernoulli random variable is $p(1-p)$, so the approximate variance of the sample mean is

$$\mathrm{Var}(\widehat{p}) \approx \frac{p(1-p)}{N_{\mathrm{trials}}}. \tag{5}$$

Replacing p with $\widehat{p}$ gives equation (2). For large N_{trials}, the central limit theorem motivates the approximate 95% interval in equation (3).